

Curso de Código Corrección de Errores
Fundación Fulgor

Trabajo Práctico 2

Código de Golay extendido

Profesores:

- Guerra, Jorge
- Pesce, Agustín
- Villar, Federico Ignacio

Estudiantes:

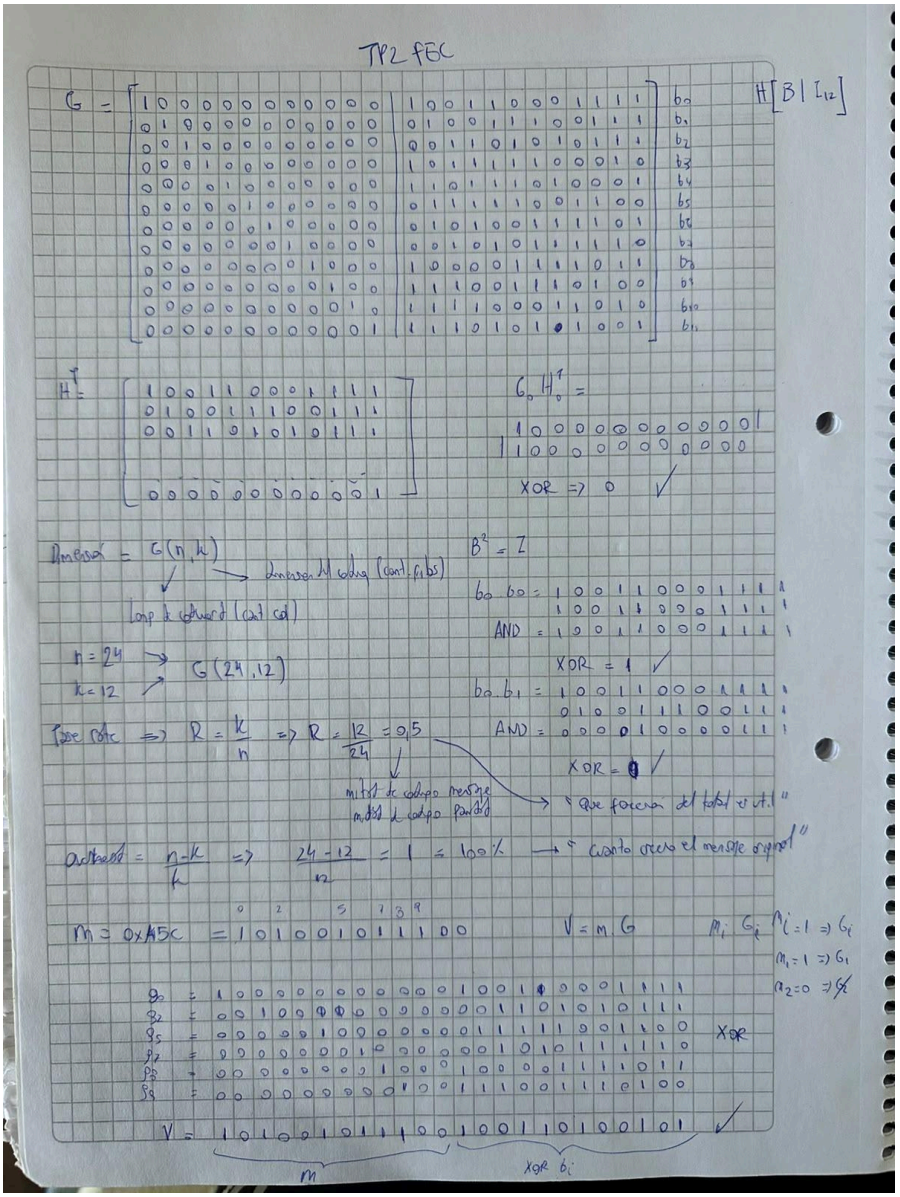
- Lugano, Damian Gabriel
- Seia, Nicolas

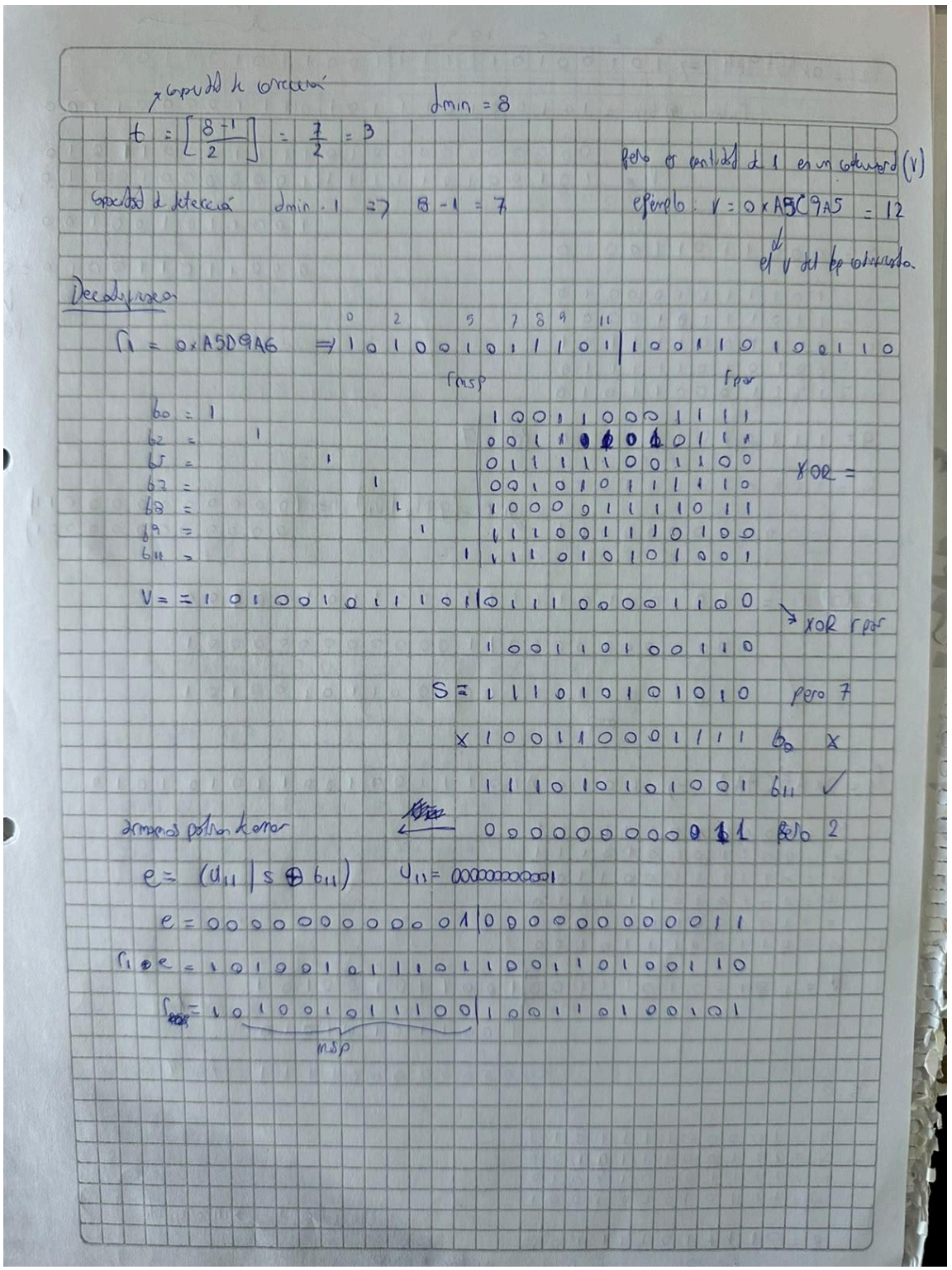
Parte A - Propiedades del código

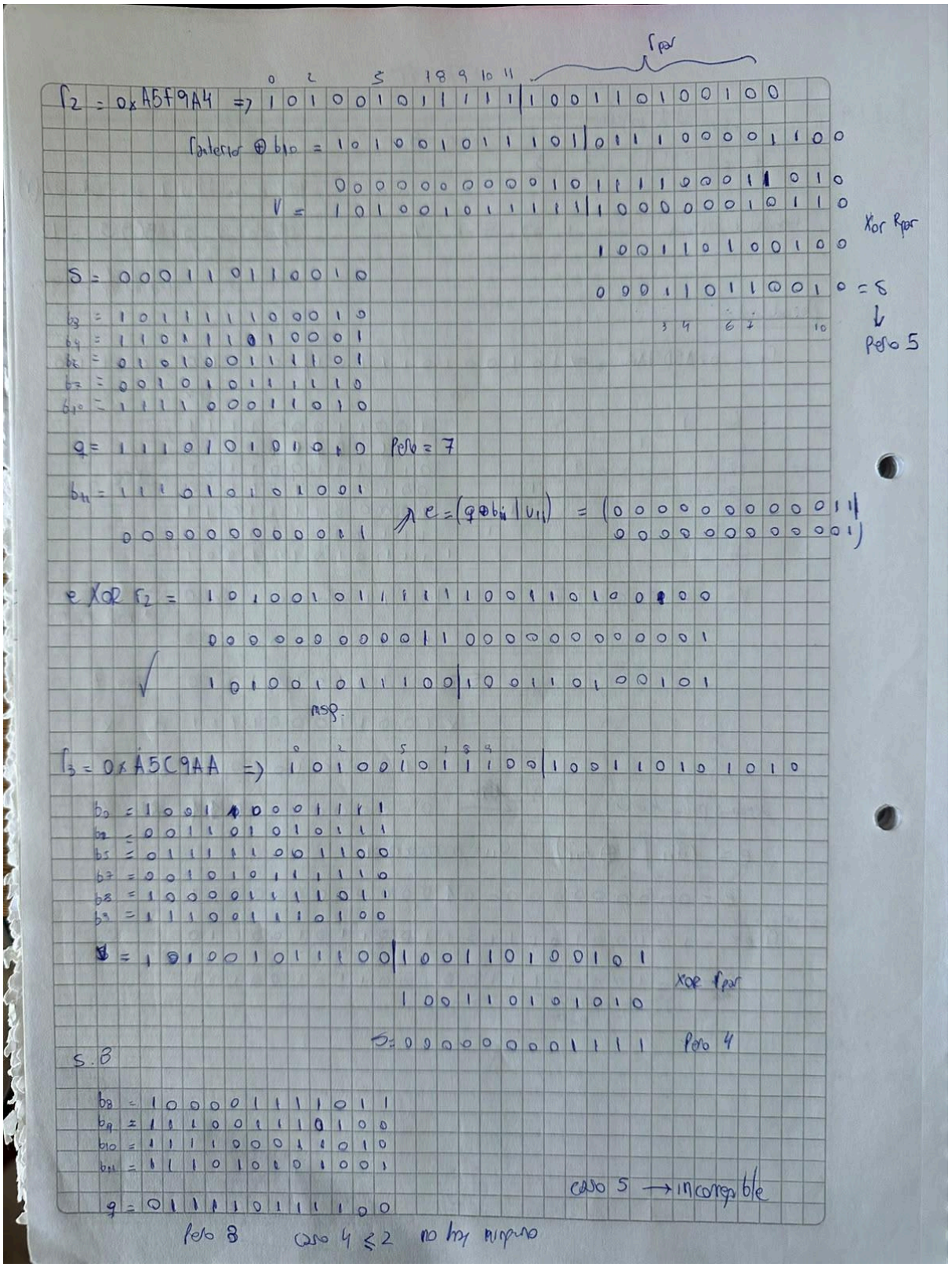
En esta sección se desarrollan los cálculos analíticos que corresponden al código Golay extendido (24,12). En base a la matriz B provista, se construye la matriz generadora G y la matriz paridad H.

Luego se aplica el algoritmo de decodificación por síndrome de cuatro casos, para decodificar manualmente las tres palabras que se disponen a resolver, indicando el síndrome de cada uno, el caso del algoritmo en el que se resuelve, el patrón de error y el mensaje decodificado.

Los cálculos completos se adjuntan a continuación:




* Capacidad de corrección
dmin = 8
t = ⌊(8+1)/2⌋ = 7/2 = 3
Capacidad de detección dmin - 1 => 8 - 1 = 7
pero el cantidad de 1 en un codeword (V)
ejemplo: V = 0xABC9A5 = 12
el V del bp corrupto.
Decodificación
r1 = 0xA5D9A6
msp
lsp
b0 = 1
b2 =
b5 =
b7 =
b8 =
b9 =
b11 =
XOR =
V = 1 0 1 0 0 1 0 1 1 1 0 1 0 1 1 1 0 0 0 0 1 1 0 0
xor rpar
1 0 0 1 1 0 1 0 0 1 1 0
S = 1 1 1 0 1 0 1 0 1 0 1 0
pero 7
x 1 0 0 1 1 0 0 0 1 1 1 1 b0 x
1 1 1 0 1 0 1 0 1 0 0 1 b11 ✓
arreglo posición de error
0 0 0 0 0 0 0 0 0 0 0 1 1 pero 2
e = (b11 | s ⊕ b11) b11 = 00000000001
e = 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 1
r1 ⊕ e = 1 0 1 0 0 1 0 1 1 1 0 1 1 0 0 1 1 0 1 0 0 1 0 1
r1 = 1 0 1 0 0 1 0 1 1 1 0 0 1 0 0 1 1 0 1 0 0 1 0 1


r_par
r2 = 0xA5f9A4 =>
r_anterior ⊕ b10 =
V =
Xor Rpar
S =
= S
Peso 5
q = Peso = 7
e = (q⊕bi | vi) =
e Xor r2 =
resp.
r3 = 0xA5C9AA =>
xor rpar
S = Peso 4
S.8
q =
caso 5 → incorreto
Peso 8 caso 4 ≤ 2 no bit mínimo

Parte B - Modelo de referencia en software

Se desarrolló el modelo de referencia en Python, partiendo de las clases *GF* y *GFPoly* construidas en el Trabajo Práctico 1. Este modelo funciona como *golden model:* sus resultados deben coincidir con los obtenidos a mano en la Parte A y con la implementación RTL de la Parte C.

Ejercicio 4 : GF(2) y álgebra matricial

Se instancio la clase *GF* con m = 1, y $P(x) = x + 1$ $(GF(1,1))$, y se verificaron las tablas de suma, producto e inverso contra la aritmética esperada de GF(2).

Se agregó el álgebra matricial faltante, producto vector-matriz, producto matriz-matriz y peso de Hamming, implementados sobre las operaciones de *GF* para mantener generalidad.

```
v = [1, 0, 1]
M = [[1, 1, 0], [0, 1, 1], [1, 0, 1]]
v*M =  [0, 1, 1]
M*M =  [[1, 0, 1], [1, 1, 0], [0, 1, 1]]
peso(v) =  2
```

Ejercicio 5 : Construcción y verificación

Se construyeron G y H en *Python* a partir de la matriz B. A su vez se verificaron que tanto $G \cdot H^T = 0$ y $B^2 = I$. Por otro lado, mediante fuerza bruta se enumeraron las 4096 palabras código, obteniendo de esta manera la distribución de pesos, la cual coincide exactamente con la provista por el enunciado.

Tabla provista:

Peso	0	8	12	16	24
Cantidad de palabras	1	759	2576	759	1

Resultados obtenidos:

```
Verificando G.H^T = 0 ...
  Resultado: True
Verificando B^2 = I ...
  Resultado: True

Enumerando 4096 palabras codigo

Distribucion de pesos obtenida:
  peso  0: 1 palabras
  peso  8: 759 palabras
  peso 12: 2576 palabras
  peso 16: 759 palabras
  peso 24: 1 palabras
```

Ejercicio 6 : Codificador y decodificador

Se implementó el codificador $v = m \,.\, G$, codificando el mensaje mediante el método *vector_matriz()*, y el decodificador de referencia, siguiendo el algoritmo de cuatro casos con la rama de no corregible. Cabe destacar que la tabla de síndromes, es decir filas de B, se deriva a partir de H, sin valores hardcodeados adicionales.

Se validó el decodificador contra r1, r2 y r3, obteniendo en los tres casos resultados idénticos a los calculados a mano en la Parte A.

Resultados obtenidos:

```
r1 resultado: {'mensaje': [1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0], 'error': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1],
'sindrome': [1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0], 'caso': 2, 'corregido': True, 'no_corregible': False}
Esperado: caso=2, sindrome peso 7, mensaje = primeros 12 bits de r1 corregido

r2 resultado: {'mensaje': [1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0], 'error': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
'sindrome': [0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0], 'caso': 4, 'corregido': True, 'no_corregible': False}
Esperado: caso=4, sindrome peso 5, q peso 7, mensaje = 0xA5C

r3 resultado: {'mensaje': None, 'error': None, 'sindrome': [0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1], 'caso': None, 'corregido': False, 'no_corregible':
 True}
Esperado: sindrome peso 4, q peso 8, no_corregible=True, mensaje=None
```

Ejercicio 7 : Caracterización del decodificador

Por último, se realizó un barrido exhaustivo de patrones de error de peso 0 a 4, clasificando cada resultado en corregido, decodificado a otra palabra código, o detectado como no corregible. Los resultados coincidieron exactamente con los valores esperados del enunciado, confirmando que el código no presenta miscorrecciones silenciosas hasta peso 3 y que todo error de peso 4 es detectado como no corregible.

Resultados finales:

```
Peso 0...
  obtenido: {'total': 1, 'corregidos': 1, 'detectados': 0, 'otra_palabra': 0}
  esperado: {'total': 1, 'corregidos': 1, 'detectados': 0, 'otra_palabra': 0}
  ¿coincide? True

Peso 1...
  obtenido: {'total': 24, 'corregidos': 24, 'detectados': 0, 'otra_palabra': 0}
  esperado: {'total': 24, 'corregidos': 24, 'detectados': 0, 'otra_palabra': 0}
  ¿coincide? True

Peso 2...
  obtenido: {'total': 276, 'corregidos': 276, 'detectados': 0, 'otra_palabra': 0}
  esperado: {'total': 276, 'corregidos': 276, 'detectados': 0, 'otra_palabra': 0}
  ¿coincide? True

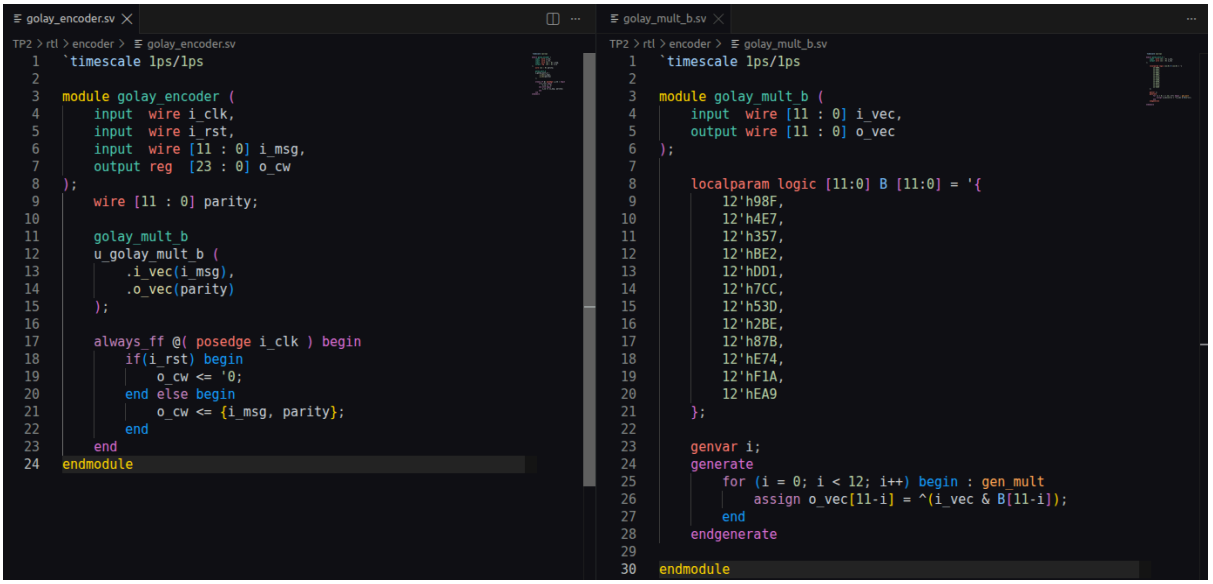
Peso 3...
  obtenido: {'total': 2024, 'corregidos': 2024, 'detectados': 0, 'otra_palabra': 0}
  esperado: {'total': 2024, 'corregidos': 2024, 'detectados': 0, 'otra_palabra': 0}
  ¿coincide? True

Peso 4...
  obtenido: {'total': 10626, 'corregidos': 0, 'detectados': 10626, 'otra_palabra': 0}
  esperado: {'total': 10626, 'corregidos': 0, 'detectados': 10626, 'otra_palabra': 0}
  ¿coincide? True
```

Parte C - Implementación RTL

Ejercicio 8 : Codificador

Para obtener la paridad del mensaje (de 12 bits), el codificador Golay realiza una multiplicación vectorial por la matriz B, resultando en un vector de paridad también de 12 bits. De este modo, la arquitectura del codificador RTL *golay_encoder* se reduce a una instancia de *golay_mult_b* seguida por el registro del mensaje original concatenado con la paridad calculada, conformando la codeword final.



golay_encoder.sv

TP2 > rtl > encoder > golay_encoder.sv

```
1  `timescale 1ps/1ps
2
3  module golay_encoder (
4      input  wire i_clk,
5      input  wire i_rst,
6      input  wire [11 : 0] i_msg,
7      output reg  [23 : 0] o_cw
8  );
9      wire [11 : 0] parity;
10
11     golay_mult_b
12     u_golay_mult_b (
13         .i_vec(i_msg),
14         .o_vec(parity)
15     );
16
17     always_ff @( posedge i_clk ) begin
18         if(i_rst) begin
19             o_cw <= '0;
20         end else begin
21             o_cw <= {i_msg, parity};
22         end
23     end
24 endmodule
```

golay_mult_b.sv

TP2 > rtl > encoder > golay_mult_b.sv

```
1  `timescale 1ps/1ps
2
3  module golay_mult_b (
4      input  wire [11 : 0] i_vec,
5      output wire [11 : 0] o_vec
6  );
7
8      localparam logic [11:0] B [11:0] = '{
9          12'h98F,
10         12'h4E7,
11         12'h357,
12         12'hBE2,
13         12'hDD1,
14         12'h7CC,
15         12'h53D,
16         12'h2BE,
17         12'h87B,
18         12'hE74,
19         12'hF1A,
20         12'hEA9
21     };
22
23     genvar i;
24     generate
25         for (i = 0; i < 12; i++) begin : gen_mult
26             assign o_vec[11-i] = ^(i_vec & B[11-i]);
27         end
28     endgenerate
29
30 endmodule
```

Por su parte, el módulo *golay_mult_b* efectúa el producto entre el vector de entrada y la matriz B, declarada internamente mediante un localparam. Dado que la matriz es simétrica, la operación implica multiplicar la entrada por cada fila mediante un operador AND lógico, aplicando posteriormente una reducción XOR que efectúa la suma módulo 2.
Aunque la matriz B se define mediante un localparam, la herramienta de síntesis descarta automáticamente las operaciones de multiplicación por cero, conservando únicamente las multiplicaciones por uno según la estructura de filas y columnas de la matriz.

Ejercicio 9 : Decodificador

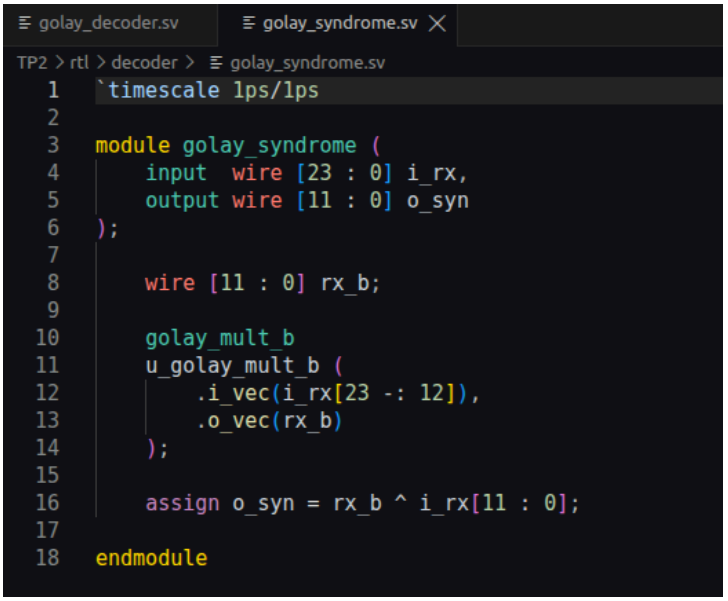
La arquitectura del decodificador RTL organiza sus submódulos dentro de una estructura en pipeline de tres etapas. Presenta un throughput de 1 ciclo e introduce una latencia fija de tres ciclos. La secuencia de procesamiento comprende las siguientes fases:

- **Cálculo del síndrome:** Obtención inicial de la señal de síndrome.
- **Cómputo para la determinación del error:** Computación de los vectores requeridos para identificar el tipo de error.

- **Generación y corrección del error:** Construcción del vector de error según el diagnóstico previo y aplicación de la corrección.

Etapa 1: Cálculo del síndrome

Esta fase inicial se focaliza de manera exclusiva en el cálculo del síndrome mediante el submódulo *golay_syndrome*, el cual evalúa $s = r[23:12] \cdot B \oplus r[11:0]$. A nivel RTL, esta operación se concreta mediante la suma módulo 2 (operador XOR) entre el producto de los 12 bits MSB del mensaje recibido y la matriz B (calculado con el bloque funcional previamente detallado) y la paridad, que son los 12 bits LSB del mensaje recibido.



En el top, este módulo es instanciado registrando tanto el síndrome calculado como el mensaje recibido. Esto garantiza que ambos datos fluyan hacia la siguiente etapa del pipeline, permitiendo procesar una nueva codeword en el ciclo inmediato posterior.

Etapa 2: Cómputo para la determinación del error

En esta fase se procesan los vectores para identificar el tipo de error y definir el error adecuado. Se contemplan 5 escenarios en total, ordenados por prioridad: 4 casos de corrección vinculados a patrones de error con pesos de 1 a 3, más un caso adicional de peso 4 que únicamente admite detección. Dichos casos se detallan a continuación:

Caso	Condición	Patrón de error
1	$w(\mathbf{s}) \leq 3$	$\mathbf{e} = (\mathbf{0} \mid \mathbf{s})$
2	$\exists\, i : w(\mathbf{s} \oplus \mathbf{b}_i) \leq 2$	$\mathbf{e} = (\mathbf{u}_i \mid \mathbf{s} \oplus \mathbf{b}_i)$
3	$w(\mathbf{q}) \leq 3$, con $\mathbf{q} = \mathbf{s} \cdot \mathbf{B}$	$\mathbf{e} = (\mathbf{q} \mid \mathbf{0})$
4	$\exists\, i : w(\mathbf{q} \oplus \mathbf{b}_i) \leq 2$	$\mathbf{e} = (\mathbf{q} \oplus \mathbf{b}_i \mid \mathbf{u}_i)$
5	ninguna de las anteriores	`o_uncorrectable`

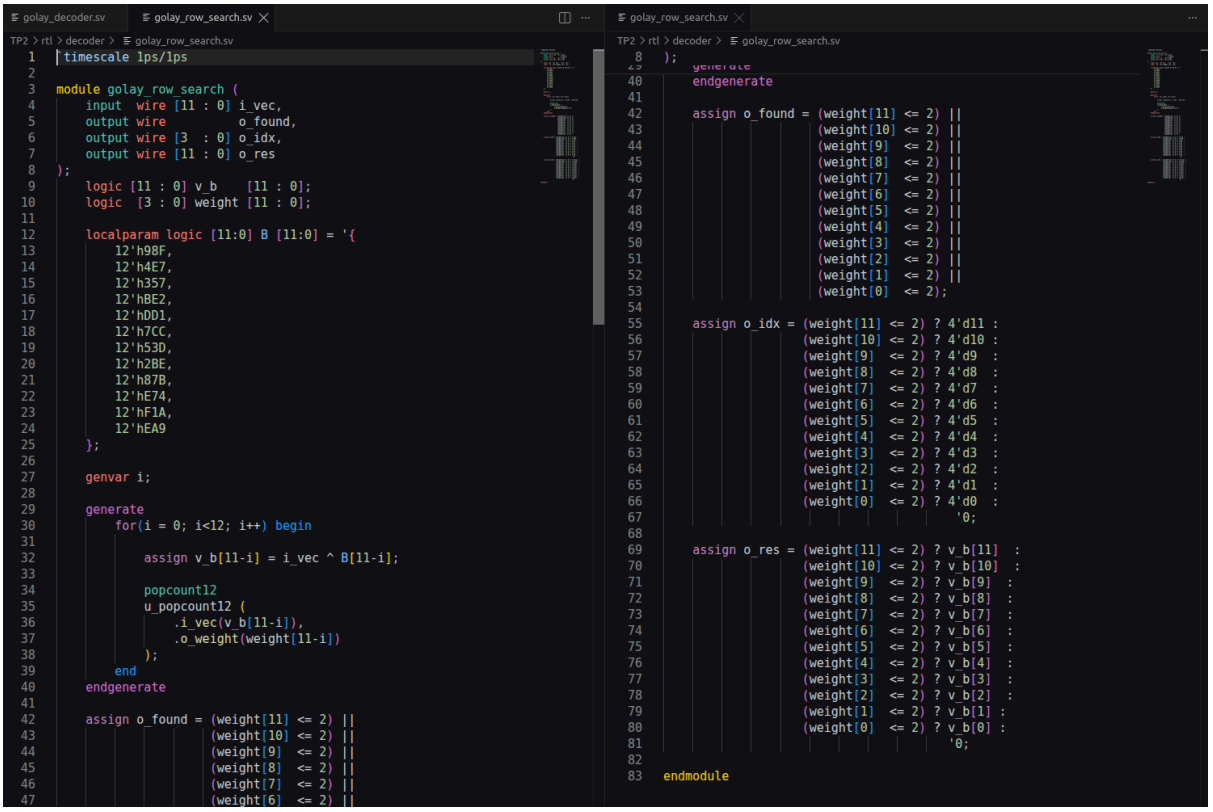
Debido a los requerimientos de la arquitectura de hardware planteada, todas las operaciones deben ejecutarse en paralelo dentro del mismo ciclo de reloj, independientemente de la prioridad asignada. La evaluación en orden jerárquico y la subsecuente corrección se llevarán a cabo en la etapa de pipeline posterior.
Para realizar el cálculo y la evaluación de los vectores se emplean dos operaciones primarias, implementadas a través de los submódulos *popcount12* y *golay_row_search*.
El submódulo *popcount12* tiene como función exclusiva determinar el peso de una palabra de 12 bits, realizando internamente la adición de cada uno de sus bits.

golay_decoder.sv | popcount12.sv

TP2 > rtl > decoder > popcount12.sv

```
1  `timescale 1ps/1ps
2
3  module popcount12 (
4      input  wire [11 : 0] i_vec,
5      output wire [3  : 0] o_weight
6  );
7
8  assign o_weight = i_vec[0]
9                  + i_vec[1]
10                 + i_vec[2]
11                 + i_vec[3]
12                 + i_vec[4]
13                 + i_vec[5]
14                 + i_vec[6]
15                 + i_vec[7]
16                 + i_vec[8]
17                 + i_vec[9]
18                 + i_vec[10]
19                 + i_vec[11];
20
21 endmodule
```

Por otra parte, el submódulo *golay_row_search* obtiene simultáneamente los doce candidatos $v \oplus b_i$ y comprueba, mediante 12 instancias de *popcount12*, si alguno posee un peso inferior a 2. En caso de concurrencia, desempata respetando la prioridad de índice. Este bloque indica si existe un candidato válido, retornando su valor e índice.



```
golay_decoder.sv
golay_row_search.sv
TP2 > rtl > decoder > golay_row_search.sv
```

```
timescale 1ps/1ps

module golay_row_search (
    input  wire [11 : 0] i_vec,
    output wire          o_found,
    output wire [3  : 0] o_idx,
    output wire [11 : 0] o_res
);
    logic [11 : 0] v_b    [11 : 0];
    logic  [3 : 0] weight [11 : 0];

    localparam logic [11:0] B [11:0] = '{
        12'h98F,
        12'h4E7,
        12'h357,
        12'hBE2,
        12'hDD1,
        12'h7CC,
        12'h53D,
        12'h2BE,
        12'h87B,
        12'hE74,
        12'hF1A,
        12'hEA9
    };

    genvar i;

    generate
        for(i = 0; i<12; i++) begin

            assign v_b[11-i] = i_vec ^ B[11-i];

            popcount12
            u_popcount12 (
                .i_vec(v_b[11-i]),
                .o_weight(weight[11-i])
            );
        end
    endgenerate

    assign o_found = (weight[11] <= 2) ||
                     (weight[10] <= 2) ||
                     (weight[9]  <= 2) ||
                     (weight[8]  <= 2) ||
                     (weight[7]  <= 2) ||
                     (weight[6]  <= 2) ||
```

```
golay_row_search.sv
TP2 > rtl > decoder > golay_row_search.sv
```

```
);
    endgenerate

    assign o_found = (weight[11] <= 2) ||
                     (weight[10] <= 2) ||
                     (weight[9]  <= 2) ||
                     (weight[8]  <= 2) ||
                     (weight[7]  <= 2) ||
                     (weight[6]  <= 2) ||
                     (weight[5]  <= 2) ||
                     (weight[4]  <= 2) ||
                     (weight[3]  <= 2) ||
                     (weight[2]  <= 2) ||
                     (weight[1]  <= 2) ||
                     (weight[0]  <= 2);

    assign o_idx = (weight[11] <= 2) ? 4'd11 :
                   (weight[10] <= 2) ? 4'd10 :
                   (weight[9]  <= 2) ? 4'd9  :
                   (weight[8]  <= 2) ? 4'd8  :
                   (weight[7]  <= 2) ? 4'd7  :
                   (weight[6]  <= 2) ? 4'd6  :
                   (weight[5]  <= 2) ? 4'd5  :
                   (weight[4]  <= 2) ? 4'd4  :
                   (weight[3]  <= 2) ? 4'd3  :
                   (weight[2]  <= 2) ? 4'd2  :
                   (weight[1]  <= 2) ? 4'd1  :
                   (weight[0]  <= 2) ? 4'd0  :
                                       '0;

    assign o_res = (weight[11] <= 2) ? v_b[11]  :
                   (weight[10] <= 2) ? v_b[10]  :
                   (weight[9]  <= 2) ? v_b[9]  :
                   (weight[8]  <= 2) ? v_b[8]  :
                   (weight[7]  <= 2) ? v_b[7]  :
                   (weight[6]  <= 2) ? v_b[6]  :
                   (weight[5]  <= 2) ? v_b[5]  :
                   (weight[4]  <= 2) ? v_b[4]  :
                   (weight[3]  <= 2) ? v_b[3]  :
                   (weight[2]  <= 2) ? v_b[2]  :
                   (weight[1]  <= 2) ? v_b[1] :
                   (weight[0]  <= 2) ? v_b[0] :
                                       '0;

endmodule
```

Con la combinación de estas dos operaciones base se estructuran los 4 casos de corrección:

- **Caso 1:** Instancia un bloque *popcount12* para medir el peso del síndrome.
- **Caso 2:** Instancia el submódulo *golay_row_search* directamente sobre el síndrome.
- **Casos 3 y 4:** Se multiplica en primera instancia al síndrome por la matriz B mediante *golay_mult_b* para generar el vector *q*. A partir de este vector, se repite el procedimiento aplicado en los Casos 1 y 2.

La estructuración de la etapa 2 en el nivel superior del RTL se ilustra a continuación:



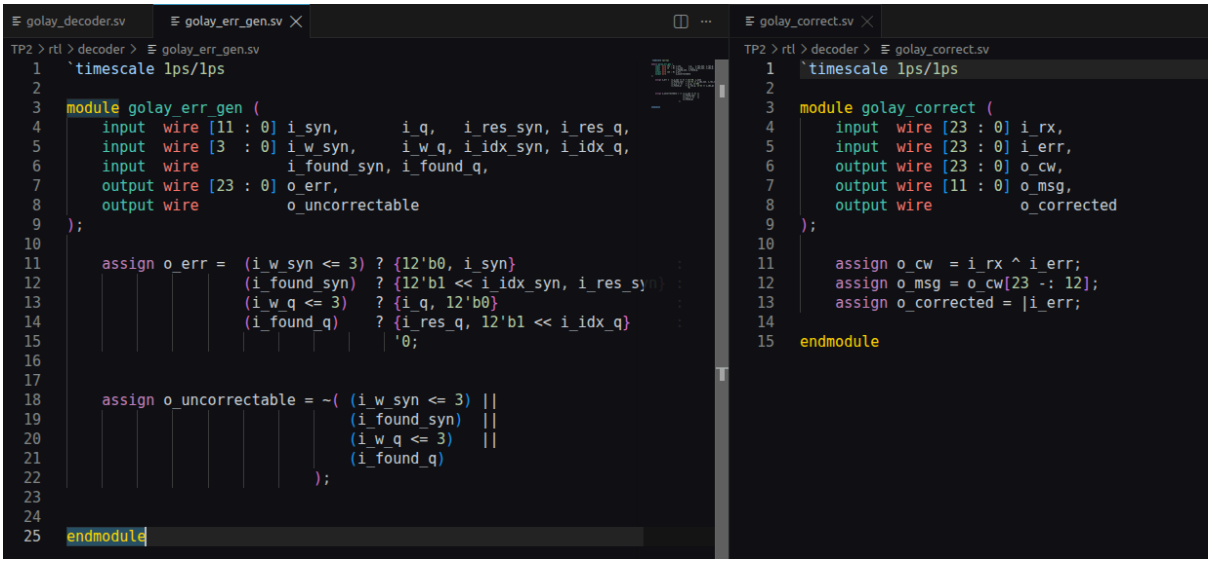
Finalmente, las salidas de estos submódulos son registradas conjuntamente con el síndrome y la palabra recibida, proveniente de la etapa 1.

Etapa 3: Generación del patrón de error y corrección

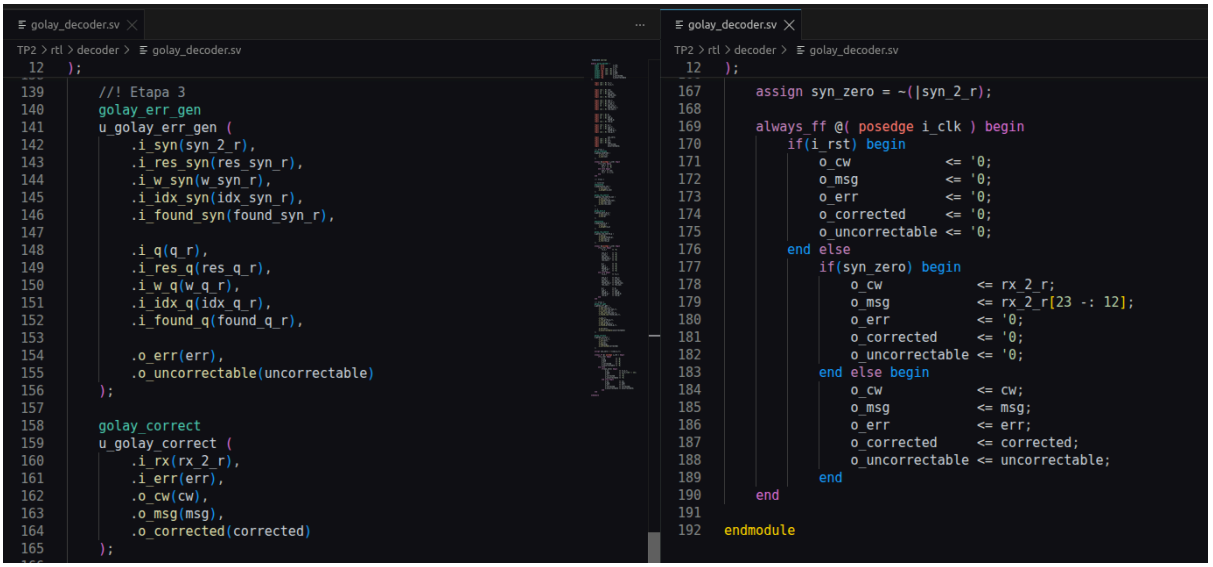
A partir de las condiciones determinadas en la fase anterior, en esta etapa se identifica la clase de error conforme a la jerarquía de prioridades establecida y se efectúa la corrección pertinente. Para llevar a cabo este proceso se dispone de dos bloques funcionales: *golay_err_gen* y *golay_correct*.

El módulo *golay_err_gen* evalúa los indicadores provenientes de la etapa 2 para seleccionar el vector de error adecuado según el orden de prioridad, o bien determinar si la palabra resulta incorregible. Esto termina infiriendo en hardware un multiplexor para cada caso con prioridad.

Por su parte, el módulo *golay_correct* aplica dicho vector de error sobre los 12 bits MSB del mensaje recibido, ejecutando la corrección mediante la operación suma en módulo 2 (XOR).



Finalmente, se registran en el top las salidas finales del módulo. En el escenario donde el síndrome es nulo (recepción de una codeword válida), la detección se realiza en esta misma etapa y se asignan directamente los 12 bits superiores recibidos como mensaje decodificado. En caso contrario, las salidas se corresponden con el resultado del procedimiento de decodificación detallado.
A continuación se ilustra la representación RTL de la etapa 3 en el top:



Ejercicio 10 : Verificación y análisis

Se desarrollaron testbenches para validar de forma exhaustiva cada uno de los submódulos del sistema, focalizando el análisis en las etapas críticas de codificación y decodificación.

En cuanto a la verificación del codificador, el testbench diseñado evaluó las 4096 palabras posibles, contrastando los resultados generados contra el modelo de referencia en Python.
Respecto al proceso de decodificación, la validación se dividió en dos fases:

- **Generador de síndrome aislada:** se comprobó primeramente que las 4096 palabras código recibidas produjeran un síndrome nulo (0).
- **Sistema de decodificación completo:** se verificó que las palabras válidas avanzaran de manera directa a la salida tras 3 ciclos, sin aplicar correcciones. Asimismo, mediante comparación directa con la referencia en Python, se confirmó la correcta corrección de patrones de error de peso 1 a 3 y la detección efectiva de los errores de peso 4.

A partir de los resultados obtenidos, se determinó que la descripción RTL reproduce fielmente el comportamiento de hardware esperado para la totalidad de los casos contemplados.

Conclusión

El desarrollo de este trabajo permitió entender y recorrer el código de Golay extendido (24,12,8) en sus tres niveles, basándonos en el cálculo analítico, modelo de referencia en software y síntesis en RTL. Verificando en cada etapa la coincidencia de resultados entre las tres partes. El hecho de realizar cada una de las partes propuestas y realizar la validación cruzada resultó fundamental para detectar errores que de otra manera hubieran pasado desapercibidos. De esta manera se logró entender y cumplir con el objetivo del trabajo, realizando un aprendizaje incremental, de etapa a etapa, entendiendo y validando su funcionamiento de manera adecuada.

Respecto a porqué el síndrome no depende del mensaje si no solo del error dado, esto se debe a que, dado $r = v + e$, el síndrome es $s = r \cdot H^T = v \cdot H^T + e \cdot H^T$. Sabiendo que, todo codeword válido cumple $v \cdot H^T = 0$, queda que $s = e \cdot H^T$, de manera tal, que el mensaje se cancela y el síndrome depende exclusivamente del patrón de error.

Finalmente, que la distancia mínima sea $d_{mín}$ = 8 y se detecten 4 errores sin corregirlos responde a que el código es originalmente de $d_{mín}$ = 7, con capacidad de corrección de hasta 3 errores, a la cual se le añadió un bit de paridad global. Esta incorporación aumenta la distancia mínima a 8, pero no altera la capacidad de corrección del código. En efecto, la corrección requiere que las esferas de radio *t* centradas en cada palabra de código no se solapen. Esto impone el límite $t = \lfloor (d_{mín} - 1) / 2 \rfloor = 3$; por ende, un patrón de error de peso 4 puede ubicarse a la misma distancia de dos palabras de código distintas, generando ambigüedad.

Repositorios

- Nicolas Seia: Repositorio_TP2
- Damian Lugano: https://github.com/dlugano32/FEC_2026/tree/main/TP2